

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



Các công nghệ lập trình hiện đại

Tìm hiểu LlamaIndex và ứng dụng RAG trong hệ thống hỏi đáp tài liệu đào tạo

GVHD: Phạm Thị Vương

SV:	Huỳnh Thanh Tiến	3123410374
	Phan Hữu Kha	3123560038
	Võ Thị Như Ý	3123410439
	Trần Hà Thanh Thùy	3123560087

TP. HỒ CHÍ MINH, THÁNG 9/2026

MỤC LỤC

1	Giới thiệu đề tài	1
1.1	Bối cảnh và động cơ	1
1.2	Mục tiêu của đề tài	1
1.3	Phạm vi báo cáo	2
1.4	Phương pháp thực hiện	2
1.5	Bố cục báo cáo	2
2	Cơ sở lý thuyết	3
2.1	Mô hình ngôn ngữ lớn và những hạn chế khi hỏi đáp tài liệu riêng . . .	3
2.2	Retrieval-Augmented Generation (RAG)	3
2.2.1	Ý tưởng	3
2.2.2	Hai pha của một hệ RAG	4
2.3	Embedding và độ tương đồng vector	4
2.3.1	Vector embedding	4
2.3.2	Độ đo cosine	4
2.3.3	Mô hình embedding tiếng Việt được chọn	5
2.4	Cắt đoạn văn bản (chunking)	5
2.4.1	Vì sao phải cắt	5
2.4.2	Các chiến lược cắt	5
2.5	Chỉ mục vector và truy hồi top- k	6
2.6	Tổng hợp câu trả lời (Response Synthesis)	6
2.7	Các khái niệm cốt lõi của LlamaIndex	7
2.8	Nhận dạng ký tự quang học (OCR)	7
2.9	Mô hình chạy cục bộ: Ollama và Qwen3	8
2.10	Đánh giá chất lượng truy hồi	8
3	Thiết lập môi trường và kiến trúc dự án	9
3.1	Công cụ cần thiết	9
3.1.1	Thư viện Python	9
3.1.2	Phụ thuộc hệ thống	9
3.2	Cài đặt và Hello World	10
3.2.1	Các bước cài đặt	10
3.2.2	Hello World	10
3.3	Cấu trúc thư mục dự án	11
3.4	Cấu hình tập trung và hai profile	11
3.5	Các script hỗ trợ	12

3.6	Mình chứng kiến trúc	13
4	Bản chất công nghệ — Bắt buộc lõi	15
4.1	Nạp dữ liệu (Ingestion)	15
4.1.1	Cơ chế	15
4.1.2	Mã nguồn	15
4.1.3	Kết quả chạy thật	16
4.1.4	Nhận xét chất lượng OCR	17
4.1.5	Vị trí trong đồ án	17
4.2	Cắt đoạn theo Điều (Node)	17
4.2.1	Cơ chế	17
4.2.2	Mã nguồn	18
4.2.3	Kết quả chạy thật	19
4.2.4	Phân tích kết quả	19
4.2.5	Hạn chế đã biết	20
4.2.6	Vị trí trong đồ án	20
4.3	Lập chỉ mục (Index)	20
4.4	Retriever và Response Synthesis	21
4.5	Tổng kết	21

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và động cơ

Quy chế đào tạo là văn bản mà sinh viên tra cứu thường xuyên trong suốt quá trình học: thời gian tối đa để hoàn thành khóa học, điều kiện dự thi, cách tính điểm học phần, điều kiện xét tốt nghiệp, thủ tục nghỉ học tạm thời hay chuyển ngành. Tại Trường Đại học Sài Gòn (SGU), các quy định này nằm trong những văn bản pháp quy dài, được ban hành dưới dạng file PDF. Một câu hỏi rất ngắn của sinh viên (ví dụ: “nghỉ học bao nhiêu buổi thì bị cấm thi?”) thường buộc người hỏi phải đọc nhiều Điều khác nhau mới tìm được câu trả lời chính xác.

Hai cách xử lý thông thường đều có hạn chế. Tìm kiếm theo từ khóa trong file PDF chỉ khớp khi người dùng dùng đúng thuật ngữ của văn bản; nếu người dùng diễn đạt khác (“cấm thi” thay cho “không đủ điều kiện dự thi”) thì không tìm thấy. Hỏi trực tiếp một mô hình ngôn ngữ lớn (LLM) thì câu trả lời nghe rất trôi chảy nhưng không có căn cứ vào quy chế của SGU, và mô hình có thể bịa ra số liệu hoặc điều khoản không tồn tại.

Kỹ thuật **RAG** (Retrieval-Augmented Generation — sinh văn bản có tăng cường truy hồi) giải quyết đúng khoảng trống này: trước khi trả lời, hệ thống tìm những đoạn quy chế liên quan nhất, rồi yêu cầu LLM chỉ trả lời dựa trên các đoạn đó. **LlamaIndex** là một framework mã nguồn mở cung cấp sẵn các thành phần cho toàn bộ chuỗi xử lý này (đọc dữ liệu, cắt đoạn, lập chỉ mục, truy hồi, tổng hợp câu trả lời), nên được chọn làm nền tảng của đồ án.

1.2 Mục tiêu của đề tài

Mục tiêu chung của đồ án là tìm hiểu LlamaIndex và vận dụng kỹ thuật RAG để xây dựng hệ thống hỏi đáp trên tài liệu đào tạo của SGU, chạy hoàn toàn với các mô hình cục bộ (không gửi dữ liệu ra dịch vụ bên ngoài). Cụ thể:

- (a) Hiểu được vai trò và cách hoạt động của từng thành phần lõi trong LlamaIndex: Document, Node, Index, Retriever, QueryEngine/Response Synthesis.
- (b) Xây dựng bước nạp dữ liệu từ PDF bản scan tiếng Việt (OCR) thành các Document có metadata.

- (c) Xây dựng bước cắt văn bản theo cấu trúc pháp lý của quy chế (từng *Điều*) thành các Node kèm số Điều.
- (d) Thiết lập kiến trúc dự án, cấu hình hai profile chạy (máy cá nhân và máy chủ GPU) và bộ câu hỏi kiểm thử để đánh giá về sau.

1.3 Phạm vi báo cáo

Báo cáo này bao gồm phần *Bắt buộc lỗi* của đề án: kiến thức nền, thiết lập môi trường, kiến trúc, và các module đã có mã nguồn chạy được. Các phần *chọn thêm* (như reranker, tinh chỉnh nâng cao) và bảng phủ kỹ thuật tổng hợp không nằm trong phạm vi tài liệu này. Ở thời điểm viết báo cáo, hai module `src/ingestion.py` và `src/node_parser.py` đã chạy được; các module lập chỉ mục, truy hồi, tổng hợp câu trả lời và chạy đánh giá **chưa có mã nguồn** và được đánh dấu rõ trong báo cáo.

1.4 Phương pháp thực hiện

Đề án làm theo từng giai đoạn nhỏ; mỗi giai đoạn phải chạy được và cho kết quả quan sát được trước khi sang giai đoạn kế tiếp. Số liệu trong Chương 3 và 4 đều lấy từ việc chạy thật các script trong repository, không dùng số liệu minh họa. Khi một module chưa tồn tại, báo cáo nói rõ là chưa triển khai thay vì mô tả như thể đã xong.

1.5 Bố cục báo cáo

- **Chương 1** — Giới thiệu đề tài (chương này).
- **Chương 2** — Cơ sở lý thuyết: LLM và hạn chế, RAG, embedding, cắt đoạn, chỉ mục vector, tổng hợp câu trả lời, các khái niệm của LlamaIndex, OCR, và độ đo đánh giá truy hồi.
- **Chương 3** — Thiết lập môi trường và kiến trúc dự án.
- **Chương 4** — Bản chất công nghệ: mã nguồn thật và kết quả chạy thật của các module lõi.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1 Mô hình ngôn ngữ lớn và những hạn chế khi hỏi đáp tài liệu riêng

Mô hình ngôn ngữ lớn (Large Language Model, LLM) là mạng nơ-ron được huấn luyện trên lượng văn bản rất lớn để dự đoán token tiếp theo. Nhờ đó LLM có khả năng hiểu câu hỏi, diễn đạt lại, tóm tắt và suy luận đơn giản. Tuy nhiên, khi dùng LLM để trả lời về một văn bản nội bộ như quy chế của SGU, có bốn hạn chế cần xử lý:

1. **Thiếu tri thức riêng.** Quy chế của một trường cụ thể, nhất là bản ban hành gần đây, hầu như không nằm trong dữ liệu huấn luyện, hoặc chỉ xuất hiện rất mờ nhạt.
2. **Tri thức bị đóng băng.** Kiến thức của mô hình dừng ở thời điểm huấn luyện; khi quy chế được sửa đổi, mô hình không tự cập nhật.
3. **Ảo giác (hallucination).** Mô hình có xu hướng sinh ra câu trả lời nghe hợp lý nhưng sai, ví dụ bịa số tín chỉ hoặc số Điều.
4. **Khó kiểm chứng.** Câu trả lời không kèm nguồn nên người dùng không biết đối chiếu ở đâu.

Có thể khắc phục bằng cách tinh chỉnh (fine-tuning) mô hình trên văn bản quy chế, nhưng cách này tốn kém, phải huấn luyện lại mỗi khi văn bản đổi, và vẫn không loại bỏ hoàn toàn ảo giác. RAG đi theo hướng khác: giữ nguyên mô hình, đưa tri thức cần thiết vào *ngữ cảnh* (prompt) tại thời điểm hỏi.

2.2 Retrieval-Augmented Generation (RAG)

2.2.1 Ý tưởng

RAG tách bài toán thành hai bước: (1) *truy hỏi* — từ kho tài liệu, tìm ra một số ít đoạn liên quan nhất tới câu hỏi; (2) *sinh* — đưa câu hỏi cùng các đoạn đó cho LLM và yêu cầu trả lời dựa trên chúng. Nếu ký hiệu q là câu hỏi, $\mathcal{D}$ là tập các đoạn văn bản trong kho, thì:

$$C = \text{Retrieve}(q, \mathcal{D}, k), \quad a = \text{LLM}(\text{prompt}(q, C)), \quad (2.1)$$

trong đó $C \subset \mathcal{D}$ là k đoạn được chọn và a là câu trả lời. Vì C lấy trực tiếp từ quy chế nên câu trả lời có căn cứ, và hệ thống có thể trích dẫn nguồn (số Điều, số trang) cho người dùng kiểm tra.

2.2.2 Hai pha của một hệ RAG

Một hệ RAG hoạt động theo hai pha có tần suất chạy khác nhau:

- **Pha chuẩn bị (offline), chạy khi tài liệu thay đổi:** nạp tài liệu $\rightarrow$ cắt thành các đoạn $\rightarrow$ biến mỗi đoạn thành vector embedding $\rightarrow$ lưu vào chỉ mục.
- **Pha truy vấn (online), chạy mỗi câu hỏi:** embedding câu hỏi $\rightarrow$ tìm k đoạn gần nhất $\rightarrow$ ghép prompt $\rightarrow$ LLM sinh câu trả lời.

Chất lượng của hệ thống bị giới hạn bởi *đoạn văn tìm được*: nếu bước truy hồi bỏ sót Điều đúng thì LLM dù tốt đến đâu cũng không thể trả lời đúng. Vì vậy phần lớn công sức của đồ án tập trung vào các bước chuẩn bị dữ liệu (nạp, cắt đoạn) và truy hồi.

2.3 Embedding và độ tương đồng vector

2.3.1 Vector embedding

Embedding là phép ánh xạ một đoạn văn bản thành một vector số thực $\mathbf{v} \in \mathbb{R}^d$ sao cho các đoạn có nghĩa gần nhau nằm gần nhau trong không gian vector. Với mô hình dạng bi-encoder, câu hỏi và đoạn văn được mã hóa *độc lập* bởi cùng một bộ mã hóa; nhờ vậy vector của toàn bộ kho tài liệu có thể tính trước và lưu lại, khi có câu hỏi chỉ cần mã hóa riêng câu hỏi.

2.3.2 Độ đo cosine

Độ tương đồng giữa vector câu hỏi $\mathbf{q}$ và vector đoạn văn $\mathbf{d}$ thường được đo bằng cosine:

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q} \cdot \mathbf{d}}{\|\mathbf{q}\| \|\mathbf{d}\|} = \frac{\sum_{i=1}^d q_i d_i}{\sqrt{\sum_{i=1}^d q_i^2} \sqrt{\sum_{i=1}^d d_i^2}}. \quad (2.2)$$

Giá trị càng gần 1 thì hai đoạn càng gần nhau về ngữ nghĩa. Khác với tìm theo từ khóa, so khớp vector cho phép “cắm thi” tìm được đoạn viết “không đủ điều kiện dự thi” vì hai cụm có embedding gần nhau.

2.3.3 Mô hình embedding tiếng Việt được chọn

Chất lượng truy hồi phụ thuộc mạnh vào việc mô hình embedding có được huấn luyện cho tiếng Việt hay không. Theo cấu hình trong `src/config.py`, đề án dự kiến dùng:

- Profile chạy cục bộ: `bkai-foundation-models/vietnamese-bi-encoder`;
- Profile máy chủ: `AITeamVN/Vietnamese_Embedding`.

Cả hai đều được nạp qua `llama-index-embeddings-huggingface`. Ở thời điểm viết báo cáo, các mô hình này mới được khai báo trong cấu hình, chưa có module nào gọi chúng để tạo embedding.

2.4 Cắt đoạn văn bản (chunking)

2.4.1 Vì sao phải cắt

Có hai lý do phải cắt tài liệu thành các đoạn nhỏ. Thứ nhất, cửa sổ ngữ cảnh của LLM có hạn, và càng nhét nhiều văn bản không liên quan thì câu trả lời càng dễ lệch. Thứ hai, một vector embedding cho cả tài liệu sẽ “pha loãng” mọi ý; mỗi đoạn nên xoay quanh một chủ đề để vector của nó đại diện đúng.

2.4.2 Các chiến lược cắt

Chiến lược	Ưu điểm	Nhược điểm
Cắt theo độ dài cố định	Đơn giản, kích thước đều	Có thể cắt ngang câu hoặc ngang điều khoản
Cắt theo câu (sentence splitter)	Không gãy câu, có chồng lấp (overlap)	Vẫn không biết ranh giới điều khoản
Cắt theo cấu trúc văn bản	Mỗi đoạn là một đơn vị nghĩa trọn vẹn	Phụ thuộc định dạng văn bản, cần quy tắc riêng

Bảng 2.1: So sánh ba chiến lược cắt đoạn.

Quy chế đào tạo có cấu trúc rất rõ: văn bản chia thành các *Điều*, mỗi *Điều* chia thành *khoản* và *điểm*. Vì vậy đề án chọn **cắt theo Điều** làm chiến lược chính; riêng những *Điều*

dài vượt ngưỡng thì cắt tiếp theo câu. Cách này giữ trọn ngữ cảnh pháp lý và cho phép gắn số Điều vào metadata để trích dẫn nguồn.

2.5 Chỉ mục vector và truy hồi top- k

Sau khi có vector cho mọi đoạn, hệ thống cần lưu chúng theo cách tìm kiếm nhanh. *Chỉ mục vector* (vector index) lưu các cặp (vector, đoạn văn, metadata). Khi có câu hỏi, bộ truy hồi (retriever) tính độ tương đồng giữa vector câu hỏi và các vector trong chỉ mục rồi trả về k đoạn có điểm cao nhất (top- k). Với kho nhỏ như một quy chế (vài chục đến vài trăm đoạn), việc so sánh vét cạn với mọi vector là đủ nhanh; các cấu trúc tìm kiếm xấp xỉ chỉ cần khi kho rất lớn.

Tham số k là sự đánh đổi: k nhỏ để bỏ sót đoạn cần thiết, k lớn đưa nhiều nhiễu vào prompt. Cấu hình hiện có trong `src/config.py` đặt `similarity_top_k = 10`.

2.6 Tổng hợp câu trả lời (Response Synthesis)

Sau truy hồi, các đoạn tìm được phải được đưa cho LLM để sinh câu trả lời. LlamaIndex gọi bước này là *response synthesis* và cung cấp nhiều chế độ (`response_mode`). Các chế độ thường gặp:

- `refine`: gọi LLM lần lượt từng đoạn, mỗi lần tinh chỉnh câu trả lời cũ bằng đoạn mới; chính xác nhưng tốn nhiều lượt gọi.
- `compact`: gộp nhiều đoạn vào cùng một prompt sao cho vừa cửa sổ ngữ cảnh rồi mới gọi LLM, giảm số lượt gọi so với `refine`.
- `tree_summarize`: tóm tắt theo cấu trúc cây, phù hợp câu hỏi tổng hợp trên nhiều đoạn.
- `simple_summarize`: cắt bớt để nhét mọi đoạn vào một lần gọi.

Đồ án đặt sẵn `response_mode = "compact"` và `streaming = True` trong cấu hình. Vì đây mới là giá trị cấu hình, báo cáo không khẳng định gì về chất lượng của chế độ này.

2.7 Các khái niệm cốt lõi của LlamaIndex

LlamaIndex mô hình hóa chuỗi RAG bằng một số đối tượng có tên rõ ràng. Bảng dưới đây tóm tắt và chỉ ra chúng nằm ở đâu trong đồ án.

Khái niệm	Vai trò	Trong đồ án
Document	Một nguồn dữ liệu thô kèm meta-data	<code>ingestion.py</code> (mỗi trang PDF một Document)
Node / TextNode	Đơn vị nhỏ hơn sinh từ Document, là đơn vị được lập chỉ mục	<code>node_parser.py</code> (mỗi Điều một Node)
Index	Cấu trúc lưu Node để truy hồi (ví dụ <code>VectorStoreIndex</code>)	Chưa triển khai
Retriever	Lấy các Node liên quan tới câu hỏi	Chưa triển khai
Response synthesizer	Ghép Node vào prompt và gọi LLM	Chưa triển khai
QueryEngine	Gộp retriever và synthesizer thành một lệnh hỏi–đáp	Chưa triển khai

Bảng 2.2: Các đối tượng chính của LlamaIndex và vị trí trong đồ án.

Điểm mạnh của thiết kế này là mỗi khâu độc lập: có thể đổi mô hình embedding, đổi cách cắt đoạn hoặc đổi chế độ tổng hợp mà không phải viết lại toàn bộ hệ thống.

2.8 Nhận dạng ký tự quang học (OCR)

Bản PDF của Quy chế đào tạo SGU là bản scan: mỗi trang là một ảnh, không có lớp văn bản (text layer). Khi đọc bằng bộ đọc PDF thông thường, hệ thống gần như không lấy được chữ nào. Để có văn bản, cần OCR.

Tesseract là công cụ OCR mã nguồn mở, có gói ngôn ngữ tiếng Việt (`vie`); trong Python được gọi qua thư viện `pytesseract`. Vì Tesseract nhận ảnh làm đầu vào, mỗi trang PDF trước hết được render thành ảnh bằng `pdf2image` (dựa trên `poppler`). Độ phân giải càng cao thì nhận dạng càng tốt nhưng chậm hơn; đồ án dùng 300 dpi.

OCR tiếng Việt luôn có sai sót, nhất là với dấu thanh và chữ in mờ (ví dụ nhầm “NHÂN” thành “NHÀN”). Điều này ảnh hưởng đến các bước sau, đặc biệt khi quy tắc cắt đoạn dựa vào việc nhận đúng từ “Điều”. Phần 4.1 sẽ nêu các lỗi cụ thể quan sát được.

2.9 Mô hình chạy cục bộ: Ollama và Qwen3

Để dữ liệu không rời máy, LLM được chạy cục bộ qua **Ollama**, một trình phục vụ mô hình mở với API HTTP (mặc định tại `http://localhost:11434`). Đồ án dự kiến dùng họ mô hình Qwen3: `qwen3:4b` cho profile chạy cục bộ và `qwen3:8b` cho profile máy chủ. LlamaIndex kết nối tới Ollama qua gói `llama-index-llms-ollama`. Thời gian chờ mặc định trong cấu hình là 180 giây do mô hình chạy trên phần cứng cá nhân có thể trả lời chậm.

2.10 Đánh giá chất lượng truy hồi

Do chất lượng RAG phụ thuộc đầu tiên vào việc tìm đúng đoạn, đồ án đánh giá riêng bước truy hồi trước khi đánh giá câu trả lời. Bộ câu hỏi kiểm thử `eval/test_questions_01.json` gồm 31 câu, mỗi câu có hai trường: `question` và `dieu_dung` (số Điều chứa đáp án). Nhờ có nhãn này, có thể dùng hai độ đo phổ biến:

- **Hit rate@ k** : tỷ lệ câu hỏi mà trong k Node đầu tiên có ít nhất một Node thuộc Điều đúng:

$$\text{Hit}@k = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\exists j \leq k : \text{dieu}(n_{i,j}) = \text{dieu_dung}_i]. \quad (2.3)$$

- **MRR** (Mean Reciprocal Rank): trung bình nghịch đảo thứ hạng của Node đúng đầu tiên:

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}. \quad (2.4)$$

Đây là kế hoạch đánh giá; module chạy đánh giá chưa được viết nên chưa có số liệu nào.

CHƯƠNG 3. THIẾT LẬP MÔI TRƯỜNG VÀ KIẾN TRÚC DỰ ÁN

3.1 Công cụ cần thiết

3.1.1 Thư viện Python

File `requirements.txt` liệt kê tên gói mà không ghi phiên bản. Bảng dưới ghi phiên bản thực tế cài trong `venv` của repository tại thời điểm đo.

Gói	Phiên bản đo được	Mục đích
<code>llama-index-core</code>	0.14.24	Framework RAG
<code>llama-index-embeddings-huggingface</code>	0.8.0	Nạp mô hình embedding
<code>llama-index-llms-ollama</code>	0.11.0	Kết nối LLM qua Ollama
<code>sentence-transformers</code>	6.0.1	Chạy embedding/reranker
<code>sentencepiece, protobuf</code>	0.2.2, không đo	Phụ thuộc của tokenizer
<code>pytesseract</code>	0.3.13	Gọi Tesseract từ Python
<code>pdf2image</code>	1.17.0	Render PDF thành ảnh
<code>gradio</code>	6.27.0	Giao diện web (dự kiến)
<code>python-dotenv</code>	1.2.3	Đọc biến môi trường từ <code>.env</code>

Bảng 3.1: Thư viện Python trong `requirements.txt`.

3.1.2 Phụ thuộc hệ thống

- **Tesseract OCR 5.5.3**, đã xác nhận có gói ngôn ngữ `vie`.
- **poppler-utils** (`pdftoppm 26.01.0`), cần cho `pdf2image`.

- **Ollama** để chạy LLM cục bộ (script `pull_models.sh` kiểm tra và báo lỗi nếu chưa cài).

3.2 Cài đặt và Hello World

3.2.1 Các bước cài đặt

Trình tự đã dùng trong dự án:

1. Tạo và kích hoạt môi trường ảo Python (venv).
2. Cài các gói Python: `pip install -r requirements.txt`.
3. Cài Tesseract (kèm gói ngôn ngữ vie) và poppler-utils bằng trình quản lý gói của hệ điều hành.
4. Cài Ollama, chạy `ollama serve`, rồi tải mô hình bằng `scripts/pull_models.sh [local|server]`.

3.2.2 Hello World

Script `scripts/hello_world.py` kiểm tra nhanh rằng LlamaIndex cài đúng: tạo một Document, dựng SummaryIndex và đếm số Document trong docstore. Script này không cần LLM hay mô hình embedding nên chạy được ngay sau khi cài thư viện.

Đoạn mã 3.1: Tạo Document và SummaryIndex với LlamaIndex (`scripts/hello_world.py`).

```
1 from llama_index.core import Document, SummaryIndex
2
3 doc = Document(text = "Điều 15. Sinh viên nghỉ học quá 20% số tiết
    sẽ không đủ điều kiện dự thi.")
4 print("Document tạo thành công!")
5 print("Nội dung:", doc.text)
6 print("ID tự sinh:", doc.doc_id)
7
8 index = SummaryIndex.from_documents([doc])
9 print("\nIndex build thành công!")
10 print("Số Document trong Index:", len(index.docstore.docs))
```

Kết quả chạy thật:

Đoạn mã 3.2: Kết quả chạy hello_world.py.

```
1 Document tạo thành công!  
2 Nội dung: Điều 15. Sinh viên nghỉ học quá 20% số tiết sẽ không đủ  
   điều kiện dự thi.  
3 ID tự sinh: 3dc12976-3b4e-4711-8798-c7d0237d98a9  
4  
5 Index build thành công!  
6 Số Document trong Index: 1
```

Ý nghĩa: doc_id được LlamaIndex tự sinh dạng UUID, và SummaryIndex lưu Document vào docstore. Câu ví dụ “Điều 15” trong script chỉ là văn bản minh họa tự viết, không phải nội dung trích từ quy chế.

3.3 Cấu trúc thư mục dự án

Đoạn mã 3.3: Cấu trúc thư mục dự án.

```
1 .  
2 |-- app/          # Giao diện Gradio (đang xây dựng, mọi co app/assets)  
3 |-- data/  
4 |   |-- raw/      # PDF gốc  
5 |   |-- ocr/      # Kết quả OCR dạng Markdown  
6 |   |-- processed/ # Dữ liệu tiền xử lý  
7 |-- docs/         # Tài liệu, giao trình  
8 |-- eval/  
9 |   |-- test_questions_01.json # 31 câu hỏi kiểm thử  
10 |   |-- results/  
11 |-- scripts/     # hello_world.py, test_ocr.py,  
   check_model_memory.py, pull_models.sh  
12 |-- src/         # config.py, ingestion.py, node_parser.py  
13 |-- storage/vector_index/ # nơi lưu chỉ mục (hiện chưa có)  
14 |-- requirements.txt
```

3.4 Cấu hình tập trung và hai profile

Toàn bộ tham số RAG nằm trong src/config.py dưới dạng một dataclass RAGConfig. Có hai profile sẵn: local (máy cá nhân, mô hình nhỏ) và server (máy chủ GPU, mô hình lớn hơn và có reranker). Profile được chọn bằng biến môi trường RAG_PROFILE, mặc định là local; giá trị không hợp lệ làm hàm get_config()

ném ValueError.

Đoạn mã 3.4: Khai báo dataclass RAGConfig trong src/config.py.

```
1 @dataclass
2 class RAGConfig:
3     embed_model_name: str
4     embed_device: str # "cpu" or "cuda"
5     llm_model_name: str
6     ollama_base_url: str
7     request_timeout: float = 180.0
8     reranker_model_name: str | None = None
9     reranker_top_n: int = 3
10    similarity_top_k: int = 10
11    response_mode: str = "compact"
12    streaming: bool = True
13    persist_dir: str = "storage/vector_index"
```

Tham số	Profile local	Profile server
Embedding	bkai-foundation- models/ vietnamese- bi-encoder	AITeamVN/Vietnamese_ Embedding
Thiết bị embedding	cuda	cuda
LLM (Ollama)	qwen3:4b	qwen3:8b
Reranker	không dùng	AITeamVN/Vietnamese_ Reranker

Bảng 3.2: Hai profile cấu hình trong src/config.py.

Một điểm cần lưu ý trong mã hiện tại: `corpus_sources` mặc định của `config.py` trỏ tới các file `quy_che_dao_tao_2021.pdf` và `qd_sua_doi_2025.pdf`, trong khi `ingestion.py` tự khai báo danh sách nguồn riêng và trong `data/raw` chỉ có 4. `QuyCheDaoTaoDHSB 2021.pdf`. Hai nơi khai báo chưa thống nhất; đây là việc cần dọn ở các bước sau.

3.5 Các script hỗ trợ

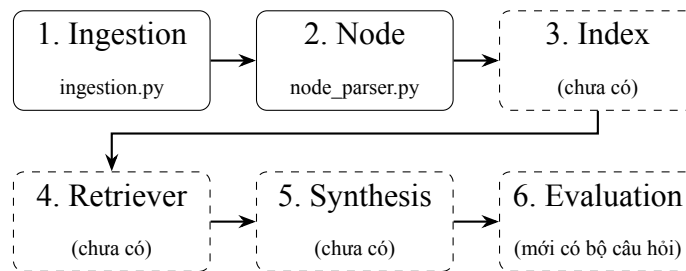
- `scripts/test_ocr.py`: render toàn bộ một PDF và OCR ra file Markdown trong `data/ocr/`, kèm số trang và tổng số ký tự. Đây là script tạo ra bản OCR

mà `node_parser.py` dùng để kiểm tra.

- `scripts/check_model_memory.py`: đo RAM/VRAM khi nạp một mô hình embedding hoặc reranker (nhận `repo_id`, `--type`, `--device`), giúp chọn mô hình vừa với phần cứng.
- `scripts/pull_models.sh`: tải LLM qua Ollama và tải embedding/reranker từ HuggingFace theo profile, đồng bộ với `config.py`; dừng với thông báo lỗi nếu thiếu `ollama`, daemon chưa chạy hoặc thiếu `huggingface_hub`.

3.6 Minh chứng kiến trúc

Chuỗi xử lý của đồ án gồm sáu giai đoạn. Hình 3.1 thể hiện chuỗi này; ô nét liền là module đã có mã chạy được, ô nét đứt là chưa có mã nguồn.



Hình 3.1: Sáu giai đoạn của pipeline RAG; nét liền: đã có mã chạy được, nét đứt: chưa triển khai.

Giai đoạn	Mức	File hiện có	Chạy được
1. Ingestion	Bắt buộc lỗi	<code>src/ingestion.py</code>	Có
2. Node	Bắt buộc lỗi	<code>src/node_parser.py</code>	Có
3. Index	Bắt buộc lỗi	không có	—
4. Retriever	Bắt buộc lỗi	không có	—
5. Response Synthesis	Bắt buộc lỗi	không có (chỉ có tham số trong <code>config.py</code>)	—
6. Evaluation	Bắt buộc lỗi	chỉ có <code>eval/test_questions_01.json</code>	Chưa có code chạy

Bảng 3.3: Trạng thái mã nguồn của từng giai đoạn.

Chưa triển khai. Index, Retriever, Response Synthesis (QueryEngine) và bộ chạy đánh giá chưa có file mã nguồn. Thư mục `storage/vector_index` hiện chưa chứa chỉ mục nào.

CHƯƠNG 4. BẢN CHẤT CÔNG NGHỆ — BẮT BUỘC LỖI

4.1 Nạp dữ liệu (Ingestion)

4.1.1 Cơ chế

Nhiệm vụ của giai đoạn này là biến file PDF quy chế thành danh sách Document của LlamaIndex. Vì PDF là bản scan nên module `src/ingestion.py` không dùng `SimpleDirectoryReader` để đọc chữ mà đi qua ba bước: (1) render từng trang thành ảnh bằng `pdf2image` ở 300 dpi; (2) OCR từng ảnh bằng `pytesseract` với `lang="vie"`; (3) đóng gói văn bản mỗi trang thành một Document kèm metadata nguồn (tên nguồn), trang (số trang) và `file_goc` (tên file).

Ngoài ra module có:

- lớp ngoại lệ `ScannedPDFLoadError` cho lỗi không tìm thấy file hoặc không render được PDF;
- hàm `has_text_layer()` thử đọc PDF bằng `SimpleDirectoryReader` để biết PDF có text layer không (hàm này định nghĩa sẵn nhưng `load_corpus` hiện chưa gọi tới);
- hàm `load_corpus()` lặp qua danh sách nguồn `CORPUS_SOURCES`, bỏ qua nguồn lỗi và ghi log thay vì dừng toàn bộ chương trình.

4.1.2 Mã nguồn

Trích hàm OCR từng trang (phần xử lý lỗi từng trang là điểm đáng chú ý: một trang lỗi chỉ bị ghi log và thay bằng chuỗi rỗng, không làm hỏng cả file):

Đoạn mã 4.1: Vòng lặp OCR từng trang trong `src/ingestion.py`.

```
1 documents: list[Document] = []
2 failed_pages: list[int] = []
3
4 for i, page in enumerate(pages, start=1):
5     try:
6         text = pytesseract.image_to_string(page, lang=lang)
7     except Exception as e:
8         logger.error(f"OCR lỗi ở trang {i} của {pdf_path}: {e}")
9         failed_pages.append(i)
```

```
10         text = ""
11
12         documents.append(Document(
13             text=text,
14             metadata={"nguồn": source_name, "trang": i, "file_goc":
15                 path.name},
16         ))
```

Hàm `load_corpus` gọi hàm trên cho từng nguồn:

Đoạn mã 4.2: Hàm `load_corpus()` nạp nhiều nguồn PDF.

```
1 def load_corpus(sources: list[tuple[str, str]] | None = None) ->
   list[Document]:
2     sources = sources or CORPUS_SOURCES
3     all_documents: list[Document] = []
4
5     for pdf_path, source_name in sources:
6         try:
7             docs = load_scanned_pdf(pdf_path, source_name)
8             all_documents.extend(docs)
9         except ScannedPDFLoadError as e:
10             logger.error(f"Bỏ qua nguồn '{source_name}' do lỗi: {e}")
11             continue
```

4.1.3 Kết quả chạy thật

Chạy `python src/ingestion.py` với file 4. QuyCheDaoTaoDHSG 2021.pdf mất khoảng 28 giây và cho kết quả:

Đoạn mã 4.3: Kết quả chạy `src/ingestion.py`.

```
1 [INFO] Hoàn tất OCR 4. QuyCheDaoTaoDHSG 2021.pdf: 17 trang, tổng
   34276 ký tự
2 [INFO] Tổng cộng nạp được 17 Document từ 1 nguồn.
```

Tức là 17 trang thành 17 Document, tổng 34 276 ký tự (trung bình khoảng 2 000 ký tự mỗi trang). Ba trang đầu được in ra làm mẫu, ví dụ trang 1 bắt đầu bằng phần quốc hiệu và tiêu đề của quyết định ban hành.

4.1.4 Nhận xét chất lượng OCR

Kiểm tra các trích đoạn in ra cho thấy OCR chưa hoàn hảo. Trang 1 nhận ra “NHÂN DÂN” và “HỒ CHÍ MINH” (đúng phải là “NHÂN DÂN”, “HỒ CHÍ MINH”), còn trang 2 có dòng “THANH PHO HO CHI MINH” mất dấu hoàn toàn. Những lỗi này nằm ở phần quốc hiệu, tiêu đề, không phải phần thân của các Điều, nhưng chúng cho thấy: (1) văn bản sau OCR cần được kiểm tra bằng mắt trước khi tin tưởng; (2) quy tắc cắt theo Điều phải chịu được lỗi OCR ở các từ khóa.

4.1.5 Vị trí trong đồ án

Đây là giai đoạn 1 của pipeline. Đầu ra là danh sách Document theo trang, mang metadata nguồn để về sau có thể trích dẫn “Quy chế đào tạo 2021, trang N ”. Một hạn chế cần ghi nhận: đầu ra theo trang có thể chia đôi một Điều nằm giữa hai trang, và hiện chưa có bước nối các trang lại trước khi cắt Điều.

4.2 Cắt đoạn theo Điều (Node)

4.2.1 Cơ chế

Module `src/node_parser.py` chuyển Document thành TextNode theo đơn vị Điều. Hàm `parse_by_dieu()` làm như sau:

1. Dùng biểu thức chính quy `DIEU_PATTERN` để tìm từng đoạn bắt đầu bằng “Điều N .” hoặc “Điều N :” và kéo dài tới ngay trước Điều kế tiếp (hoặc hết văn bản).
2. Với mỗi đoạn, trích số Điều bằng `extract_dieu_number()` và gắn vào `metadata['dieu']`, giữ nguyên metadata gốc của Document.
3. Nếu số từ của Điều (đếm gần đúng bằng tách theo khoảng trắng) vượt `max_chunk_tokens` (mặc định 1024), cắt tiếp bằng `SentenceSplitter` với `chunk_overlap = 50` và gắn thêm `metadata['phan']`.
4. Document không khớp mẫu nào được giữ nguyên thành một Node với `dieu = "khong_co_dieu"`, đồng thời ghi cảnh báo.

Hàm `validate_nodes()` thống kê chất lượng: tổng số Node, số Node không rõ Điều, tỷ lệ và độ dài trung bình.

4.2.2 Mã nguồn

Biểu thức chính quy và hàm trích số Điều:

Đoạn mã 4.4: Biểu thức chính quy tách Điều và hàm trích số Điều.

```
1 DIEU_PATTERN =  
    re.compile(r"(Điều\s+\d+[.:.]*?) (?:Điều\s+\d+[.:.]|\Z)",  
    re.DOTALL)  
2 DIEU_NUMBER_PATTERN = re.compile(r"Điều\s+(\d+)")  
3  
4 def extract_dieu_number(text: str) -> str:  
5     """Trích xuất số thứ tự của Điều từ đoạn văn bản."""  
6     match = DIEU_NUMBER_PATTERN.match(text.strip())  
7     return match.group(1) if match else "khong_ro"
```

Giải thích biểu thức: `Điều\s+\d+[.:.]*` khớp phần đầu Điều (từ “Điều”, khoảng trắng, chữ số, rồi dấu chấm hoặc hai chấm); `. * ?` lấy nhỏ nhất có thể (lazy) với cờ `DOTALL` để qua nhiều dòng; `(?=\dots|\Z)` là lookahead dừng ngay trước Điều kế tiếp hoặc cuối văn bản. Yêu cầu có dấu chấm/hai chấm sau số là điều kiện quan trọng: nhờ đó các tham chiếu chéo kiểu “theo quy định tại Điều 5 của Quy chế này” không bị nhầm thành tiêu đề Điều mới.

Phần lõi của hàm cắt:

Đoạn mã 4.5: Vòng lặp cắt Node theo Điều trong `parse_by_dieu()`.

```
1 for dieu_text in matches:  
2     dieu_so = extract_dieu_number(dieu_text)  
3     base_metadata = {"dieu": dieu_so, **doc.metadata}  
4     estimated_tokens = len(dieu_text.split())  
5  
6     if estimated_tokens <= max_chunk_tokens:  
7         all_nodes.append(TextNode(text=dieu_text.strip(),  
            metadata=base_metadata))  
8     else:  
9         sub_texts = sub_splitter.split_text(dieu_text)  
10        for j, sub_text in enumerate(sub_texts):  
11            all_nodes.append(  
12                TextNode(  
13                    text=sub_text,  
14                    metadata={**base_metadata, "phan": j + 1},  
15                )  
16        )
```

4.2.3 Kết quả chạy thật

Khởi `__main__` của `node_parser.py` đọc bản OCR đã lưu `data/ocr/4.QuyCheDaoTaoDHSB 2021.md` (không phải đầu ra trực tiếp của `ingestion.py`), bọc thành một Document rồi cắt. Kết quả chạy thật:

Đoạn mã 4.6: Kết quả chạy `node_parser.py`.

```
1 - tong_so_node: 24
2 - node_khong_ro_dieu: 0
3 - ty_le_khong_ro: 0.0%
4 - do_dai_trung_binh_ky_tu: 1400
5
6 [Node 1] - Metadata: {'dieu': '1', 'source': '4. QuyCheDaoTaoDHSB
    2021.md', 'doc_type': 'quy_che'}
7 [Node 2] - Metadata: {'dieu': '2', ...}
8 [Node 3] - Metadata: {'dieu': '3', ...}
```

Có 24 Node, không Node nào bị gán “không rõ Điều” (0%), độ dài trung bình 1 400 ký tự.

4.2.4 Phân tích kết quả

Kiểm tra thêm bằng cách in danh sách nhãn `dieu` của toàn bộ 24 Node cho thấy:

- Văn bản OCR gồm hai phần nối nhau: *Quyết định ban hành* (3 Điều) và *Quy chế* (21 Điều). Do đó chuỗi nhãn là 1, 2, 3 rồi 1, 2, ..., 21: **ba nhãn Điều 1, 2, 3 xuất hiện hai lần** (một lần của Quyết định, một lần của Quy chế).
- Trong toàn bộ 24 Node không có Node nào bị cắt nhỏ (không có `phan`), tức không Điều nào vượt ngưỡng 1024 từ. Điều dài nhất khoảng 3 238 ký tự.
- Có 36 lần xuất hiện cụm “Điều *N*” trong văn bản nhưng chỉ 24 lần là tiêu đề Điều; 12 lần còn lại chủ yếu là tham chiếu chéo (không có dấu chấm sau số) và đã không bị tách nhầm, đúng như thiết kế.

Hệ quả cần xử lý sau: vì hai Điều mang cùng nhãn (ví dụ Điều 2 của Quyết định và Điều 2 của Quy chế) nên trường `dieu` một mình không phân biệt được chúng. Khi

chấm truy hồi bằng `dieu_dung` trong bộ câu hỏi, Node của Quyết định ban hành có thể bị tính nhầm là đúng. Cách khắc phục hợp lý là bổ sung metadata phân biệt phần (ví dụ tên văn bản hoặc chương) khi cắt.

4.2.5 Hạn chế đã biết

- Số “token” được đếm bằng số từ tách theo khoảng trắng, chỉ là xấp xỉ, không phải token của mô hình.
- Nếu OCR làm mất dấu chấm sau số Điều (ví dụ “Điều 12” không có dấu chấm) thì tiêu đề đó không khớp mẫu và Điều bị gộp vào Điều trước.
- Ngược lại, một câu tham chiếu kết thúc bằng dấu chấm ngay sau số Điều (“...quy định tại Điều 5.”) có thể bị nhận nhầm là tiêu đề mới. Trong văn bản hiện có, trường hợp này chưa xảy ra.
- Kiểm thử mới chạy trên một văn bản; chưa có `scripts/test_node_parser.py` và chưa có kiểm thử tự động.

4.2.6 Vị trí trong đồ án

Đây là giai đoạn 2 của pipeline. Mỗi Node là một Điều với nhãn `dieu`, làm đơn vị cơ bản để lập chỉ mục và để trích dẫn nguồn khi trả lời.

4.3 Lập chỉ mục (Index)

Về mặt lý thuyết (xem Chương 2), bước này sẽ embedding từng Node bằng mô hình tiếng Việt đã chọn trong profile, lưu vào `VectorStoreIndex` và lưu xuống thư mục `storage/vector_index` (giá trị `persist_dir` trong `config.py`) để không phải embedding lại mỗi lần chạy.

Chưa triển khai. Module Index chưa triển khai: không có `src/index_builder.py`, và thư mục `storage/vector_index` hiện không chứa chỉ mục nào. Mô tả trên là thiết kế dự kiến theo cấu hình, chưa được kiểm chứng bằng cách chạy.

4.4 Retriever và Response Synthesis

Khi có chỉ mục, retriever sẽ lấy top- k Node (cấu hình hiện là $k = 10$) và bộ tổng hợp câu trả lời sẽ ghép chúng vào prompt để LLM sinh câu trả lời ở chế độ compact, có streaming. Câu trả lời cần kèm nhãn điều của các Node nguồn để người dùng kiểm chứng.

Chưa triển khai. Chưa có `src/retriever.py` và `src/query_engine.py`. Hiện chỉ tồn tại các tham số cấu hình `similarity_top_k`, `response_mode`, `streaming` trong `src/config.py`. Phần rerank (`reranker_model_name`, `reranker_top_n`) thuộc nhóm chọn thêm nên không trình bày trong báo cáo này.

4.5 Tổng kết

Tại thời điểm viết báo cáo, hai giai đoạn đầu của pipeline đã chạy được trên Quy chế đào tạo SGU 2021: OCR ra 17 Document (34 276 ký tự) và cắt theo Điều ra 24 Node với 0% Node không rõ Điều. Đồng thời phân tích cho thấy hai việc cần làm tiếp: thống nhất nguồn dữ liệu giữa `config.py` và `ingestion.py`, và bổ sung metadata phân biệt hai văn bản trong cùng một file (Quyết định ban hành và Quy chế) vì nhãn Điều 1–3 bị trùng. Các giai đoạn Index, Retriever, Response Synthesis và Evaluation là phần còn lại phải xây dựng.